

(Optional) HTML Pages with a Database

The same student data from Lesson 4, now as HTML pages.

Learning Goals

- Show database records as HTML pages next to the JSON API, using the same model and table.
- Explain which files the script reuses from Lesson 4 and which are new.
- Use CSRF tokens, escaping, and the hidden `_method` field in HTML forms.

What Is Different?

`run1-4.sh` builds a JSON **API**. `6-2/run1-6-2.sh` builds the same API **plus HTML pages** that you open in a browser.

	Lesson 4 (<code>run1-4.sh</code>)	This lesson (<code>run1-6-2.sh</code>)
Routes file	<code>routes/api.php</code>	<code>routes/api.php</code> + <code>routes/web.php</code>
Controller	<code>Api/StudentController</code> (JSON)	+ <code>StudentController</code> (HTML)
Client	<code>curl</code>	Browser
Database	<code>studentdb</code>	<code>studentdb</code> (same)

- Everything before Step 5 (project, MySQL, `.env`) is the same as `run1-4.sh` . See `4-2` .

What the Script Copies

```
LESSON4_DIR="$SCRIPT_DIR/../../4. Models and Database/student-api1-4"

# Same finished files as Lesson 4
for f in app/Http/Controllers/Api/StudentController.php \
        app/Models/Student.php routes/api.php \
        database/migrations/..._create_students_table.php \
        database/seeder/StudentSeeder.php
do cp "$LESSON4_DIR/$f" "$f"; done

# New in this lesson: the HTML part
for f in app/Http/Controllers/StudentController.php routes/web.php
do cp "$SOURCE_DIR/$f" "$f"; done
```

- The model, table, and seeder are **not** duplicated: the script reuses the Lesson 4 files.
- So the `4. Models and Database` folder must exist. The script stops with a message if it does not.

Same Database, Two Kinds of Pages

URL	Returns	Controller
<code>/api/students</code>	JSON	<code>Api\StudentController@</code>
<code>/students</code>	HTML list	<code>StudentController@inde</code>
<code>/students/create</code>	HTML form	<code>StudentController@crea</code>
<code>/students/{student}</code>	HTML details	<code>StudentController@show</code>

- Both controllers use the same `Student` model and the same `students` table.
- A student added in the HTML form shows up in

Two Things Every HTML Form Needs

```
<form method="POST" action="/students">' . csrf_field() . '
```

- `csrf_field()` adds a hidden **CSRF token**. Laravel rejects a web `POST`, `PUT`, or `DELETE` without it with **419 Page Expired**. API routes do not need it.
- `e($student->name)` escapes user text. A name like `<script>` is shown as text and does not run in the browser.
- HTML forms can only send `GET` and `POST`. The hidden field `_method` (`PUT` or `DELETE`) tells Laravel which method to use.

Run and Test

```
cd "6. View and Controllers/6-2"  
bash ./run1-6-2.sh  
cd student-api  
php artisan serve --port=8080  
# WSL2: php artisan serve --host=0.0.0.0 --port=8080
```

Open these pages in a browser:

1. `http://localhost:8080/` : the home page with links.
2. `/students` : the student list. Click **Add New Student**, fill in the form, and submit.
3. The new student appears in the list. Then open `/api/students` to see the same record as JSON.
4. Open a student, click **Edit**, change the major, and save. Then click **Delete**.

Why This Part Is Optional

The HTML pages are extra. You can skip them.

- The rest of this module builds APIs, so Lessons 1 to 5 are enough to continue.
- Use this part when you want to see the same MVC flow with a browser: route, controller, model, and HTML response.
- Like `run1-4.sh`, it never uses `migrate:fresh`, so the Module 1 data in `studentdb` is kept.
- Next: **6-3** replaces the HTML strings with Blade templates.